EX25 · Vectors, linear systems and numerical accuracy

Physical foundations, mathematical derivations, hand calculations, conditioning and engineering interpretation.

LANGUAGE / ID	UNIT	HANDOUT
EN · EX25	algebra / vectors and linear systems	Engineering & Mathematics Theory Guide

1 · What engineering problem are we solving?

Suppose two actuators push a small platform in a plane. Their directions are fixed, but we need the magnitudes that will create a specified resultant force. Related problems arise in vehicle control, load sharing, mechanisms and sensor calibration.

A force of magnitude F and unit direction $d=(d_x,d_y)$ is represented by vector Fd. Summing two actuator contributions yields the force balance. We neglect moments, actuator limits and dynamics: this is a **linear, static** model.

$$\mathbf{F} = u_1\hat{\mathbf{d}}_1 + u_2\hat{\mathbf{d}}_2$$

2 · From physical model to matrix

Each force has horizontal x and vertical y components. The actuator unit directions are $d_1=(0.8,0.6)$ and $d_2=(0.6,-0.8)$. We want resultant $b=(14,2)$ N.

```
Fx = 0.8*u1 + 0.6*u2 = 14 N
Fy = 0.6*u1 - 0.8*u2 =  2 N

         |  0.8   0.6 |   |u1|   |14|
A @ u =  |  0.6  -0.8 | @ |u2| = | 2|
```

Each **column** of A corresponds to one actuator. Each **row** corresponds to one force component. Adding actuators adds columns; adding force-balance components adds rows.

3 · Vector refresher: magnitude and dot product

$$\|\mathbf{v}\|_2 = \sqrt{v_x^2 + v_y^2}$$

The norm expresses vector length. For a force it has units of newtons. For direction d_1: $\sqrt{(0.8^2+0.6^2)}=\sqrt{1}=1$, so it is a unit vector.

$$\mathbf{a}\cdot\mathbf{b} = a_x b_x + a_y b_y$$

The dot product checks orthogonality of unit directions: $0.8\cdot0.6+0.6\cdot(-0.8)=0$. Our actuator directions meet at 90 degrees, a particularly favorable numerical geometry.

4 · Hand-worked example: a 2×2 system

Before programming, solve a simpler system. Find x and y such that:

```
3*x + 1*y = 9
1*x + 2*y = 8
```

Substitution: the second equation gives x=8−2y. Substituting into the first gives 3(8−2y)+y=9, or 24−5y=9, hence y=3 and x=2.

Validation: 3·2+1·3=9 and 1·2+2·3=8. The exact solution is (2,3); numerical linear algebra should return the same result up to floating-point accuracy.

5 · Determinant and invertibility

For a 2×2 matrix A=[[a,b],[c,d]], the determinant is ad−bc. A zero determinant means that the equations cannot determine a unique solution: the matrix is **singular**.

$$\det(\mathbf{A}) = ad - bc$$

In the hand example, det(A)=3·2−1·1=5 ≠ 0. In the actuator system, det(A)=0.8·(−0.8)−0.6·0.6=−1. Both are uniquely solvable.

Solving with numerical matrix factorizations (as **np.linalg.solve** does) is preferable to explicitly forming A^{-1}. Solving and matrix inversion are mathematically connected but not identical computational procedures.

6 · Hand calculation for our physical case

Since A has orthonormal columns, $A^TA=I$. Multiplying Au=b by A^T gives $u=A^Tb$. This special simplification does not hold for a general matrix.

$$\mathbf{A}^T\mathbf{A} = \mathbf{I}$$
$$\mathbf{u} = \mathbf{A}^T\mathbf{b}$$

```
u1 = 0.8*14 + 0.6*2 = 12.4 N
u2 = 0.6*14 - 0.8*2 =  6.8 N

Fx = 0.8*12.4 + 0.6*6.8 = 14 N
Fy = 0.6*12.4 - 0.8*6.8 =  2 N
```

Both magnitudes are positive, so they are compatible with push-only actuators in this particular case. Other targets could require negative forces or exceed realistic actuator capacity.

7 · Computational versus modeling error

The **residual** measures the error when reconstructing the target using the computed solution. If r=Au−b, absolute residual is $||r||_2$ (N); relative residual is $||r||_2/||b||_2$ (dimensionless for b≠0).

$$\rho = \frac{\|\mathbf{Au} - \mathbf{b}\|_2}{\|\mathbf{b}\|_2}$$

A small residual shows internal consistency among the numerical data, the model and the computed solution. It **does not prove** that the model includes all real-world effects (torques, friction, uncertainty, saturation or calibration errors). Nor does it guarantee that the solution is insensitive to data errors.

8 · Conditioning and perturbation amplification

Assume a small measurement error δb changes the target. Solving A(u+δu)=b+δb gives a change δu. For nonsingular A, the 2-norm condition number bounds the ratio of relative solution and input changes.

$$\frac{\|\delta \mathbf{u}\|_2}{\|\mathbf{u}\|_2} \leq \kappa_2(\mathbf{A}) \frac{\|\delta \mathbf{b}\|_2}{\|\mathbf{b}\|_2}$$

$\kappa_2(A) \geq 1$ for invertible matrices. Larger values permit much stronger sensitivity. Our orthogonal matrix has $\kappa_2(A)=1$, so it does not amplify relative target perturbations.

```
delta_b = [0.000, 0.010] N
u_original  = [12.400, 6.800] N
u_perturbed = [12.406, 6.792] N
delta_u     = [ 0.006,-0.008] N

||delta_b|| = 0.010 N
||delta_u|| = 0.010 N
relative change ~ 0.07071% in both.
```

9 · Almost identical equations

Consider A=[[1,1],[1,1.000001]], u=(10,5), and b=(15,15.000005). Changing only the second component of b by one millionth makes the new solution approximately (9,6). A tiny input perturbation leads to an order-one change in u.

```
A_bad = [[1, 1], [1, 1.000001]]
u       = [10, 5]
b       = [15, 15.000005]
new_b   = [15, 15.000006]
new_u   = [9, 6]

cond_2(A_bad) ~ 4,000,002
```

Both equations carry almost the same information. The determinant is 10^{-6} and the system is badly conditioned. The residual can remain tiny while a reliable estimate of u requires extremely accurate inputs.

10 · Floating-point arithmetic

Python/NumPy usually represent real-valued calculations with **float64**, which gives roughly 15–16 significant decimal digits, not exact real numbers. Sums, differences and products are rounded to representable binary values.

```
0.1 + 0.2 == 0.3    # False
np.isclose(0.1 + 0.2, 0.3)  # True
```

Distinguish **round-off error** (representation and arithmetic), **truncation error** (approximations such as Euler's method), and **modeling error** (idealized physics). Good numerical work assesses them separately.

11 · Engineering interpretation of outputs

Output	Engineering meaning
u1, u2 [N]	Commanded force magnitude of each actuator
Au [N]	Model-reconstructed resultant force
Absolute residual [N]	Force mismatch in physical units
Relative residual [–]	Mismatch normalized by the target magnitude
$\kappa_2(A)$ [–]	Intrinsic sensitivity of actuator geometry
Relative change [–]	Effect of a particular target perturbation

12 · Summary and code connection

- The **matrix** encodes geometry and physics; it is not merely a table of numbers.
- **solve_forces** converts force targets into actuator commands.
- **residual_norm** and **relative_residual** verify numerical consistency.
- **sensitivity_analysis** separates numerical accuracy from robustness to perturbed inputs.
- **Tests** check numerical outputs, shapes and failures before we trust an implementation.

> **Core lesson:** an engineering computation does not end when it produces numbers. We must justify the physical model, validate units and quantify the reliability of our results.